

Cohort module three, two hours

Route the Right Context to Every Task

The session where the instructions file gets taken apart. Everyone leaves with a router they can test, and two skills lifted out of prompts they were rewriting by hand.

0:00 to 0:20

An autopsy on your instructions

Goal Everyone has sorted their own instructions into three piles and seen how little of it any one task needs.

Say

- Give them the three piles and let them do the sorting themselves: facts about this repository, ways of working, and things that apply to one task only.
- Ask for a show of hands on how much landed in the third pile. It is usually most of it, and seeing that is the argument.
- Name what each pile becomes before moving on, so the rest of the session has somewhere to put things.

Run

```
$ wc -l <their instructions file>
```

Room does Annotates their own file with three markers and reports the split.

Ask *Which line in there was added after one bad result and never revisited?*

0:20 to 1:05

Assembling context as a calculation

Goal Each person has a function that turns a task into its material, and a test that pins what it returned.

Say

- Derive the inputs with the room rather than presenting them. What does the task say it will touch, who owns those paths, what rules apply there.
- Build the lab's router live and keep it small. If it needs a diagram it has become a framework.
- Write the test immediately after, because the point of code over prose is that it can be asserted. Give the router a task and pin what came back.

Run

```
$ node steps/03-context/demo.ts route 12
$ node steps/03-context/demo.ts route 31
```

The router is `steps/03-context/router.ts`.

Room does

- Sees two tasks receive visibly different material, with the reason for each inclusion.
- Writes a first router against their own ownership graph.

Ask *What did your router include that the task did not need? Remove it and say what broke.*

1:05 to 1:40

Lifting a procedure out of a prompt

Goal Everyone has written one skill that states an outcome and its evidence, without scripting the keystrokes.

- Say**
- Ask what they have retyped more than three times this month. That is the first skill, and they already know what it is.
 - Show the failure mode before the good version: a long procedure that is worse than the agent's own judgement and goes stale within a month.
 - Give the three parts and hold them to it. The outcome, the constraints that are not obvious from the code, and what proves it is done.
-

Run

```
$ ls steps/03-context/skills
$ cat steps/03-context/skills/change-the-shared-contract.md
```

- Room does**
- Reads one worked example, then closes it and writes their own.
 - Swaps with a neighbour to check the outcome is stated clearly enough to act on.
-

Ask *Could this skill be copied to a different repository unchanged? If not, some of it is context.*

1:40 to 2:00

The shared definition trap

Goal They understand why a naive mapping of files to areas ships a break, and have the dependency edge that fixes it.

Say

- Pose it as a puzzle. One file changed, one area owns it, one suite ran, everything green, production broken. What happened?
- Draw out the edge: three areas are built against that definition, so all three are affected.
- Note the second use of the same graph. It also tells the router which rules a task needs, so this is one calculation serving two purposes.

Run

```
$ node target/tools/affected.mjs --since HEAD~1
```

Room does Sees one edited definition select three areas and three different check commands.

Ask *Where is your shared definition, and how many places restate it today?*

1:35 to 1:50

What it read before it decided

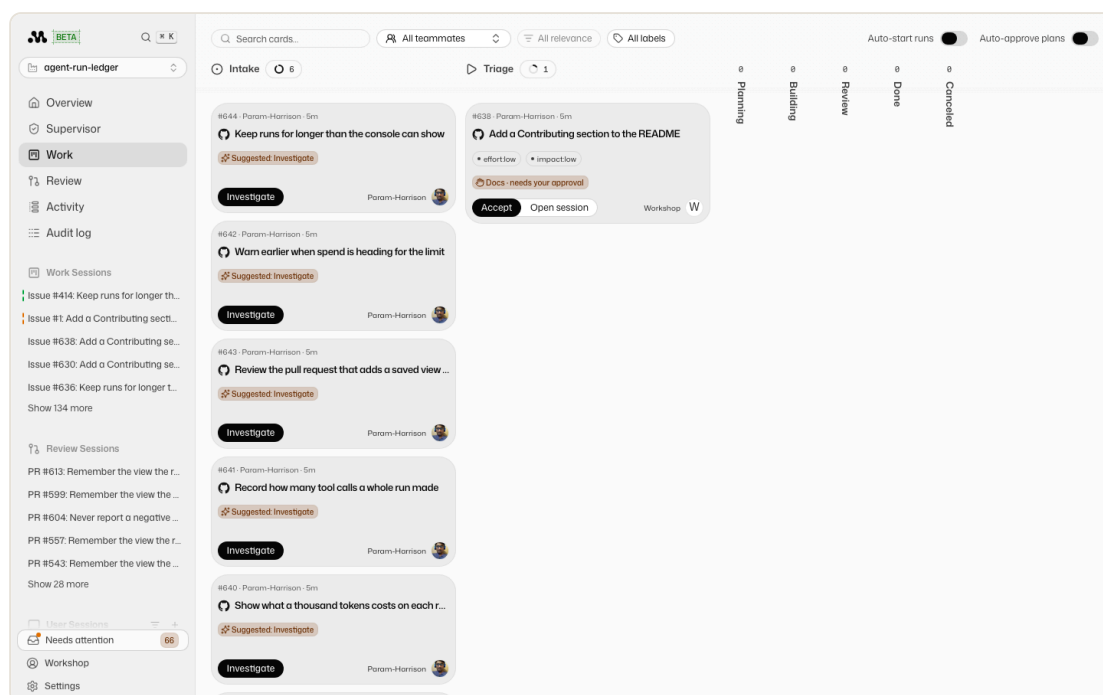
Goal They can tell an assessment grounded in the repository from one grounded in the issue text.

Say

- Read the file paths in the assessment and check every one exists.
- An assessment that names line numbers has opened the file; one that paraphrases the issue has not.
- This is the cheapest quality signal available before any code is written, and it costs one read.

Run

```
$ make status
```



The assessment, with the files it names and the confidence it claims.

Show on the board

Work to a triaged card to its session to the assessment table

Every path it names, check one

An assessment that cites a file and a line number has opened the file. One that restates the issue in different words has not, and the two read alike until somebody checks.

The best outcome is sometimes no change

One run here answered a request for a cost column by proving the column already existed, with line numbers and five real currency values read from a live service. The issue was wrong and the run was right.

Confidence is a claim, not a feeling

The table states high or low confidence. Ask what evidence would have to be missing for that word to be wrong, and whether the run went and looked for it.

Room does

- Verifies two cited paths against the repository.
- Marks the assessment as grounded or not, and says which evidence decided it.

Ask

Which sentence in this assessment could have been written without opening the repository?

Concepts: the router, writing a skill, why prompts grow, affected areas.